

Nirapod

Viva reference: what each member built, and where it lives

Project	Nirapod - A women safety app
Course	CSE470 - Software Engineering
Group	Team 7, Section 4
Semester	Summer 26
Stack	MongoDB, Express, React, Node - strict MVC on the server
Scale	26 functional requirements, 130 source files, 295 automated tests

How to use this document

Part One walks every feature in the contribution tracker, in sprint order, and names the exact files and functions behind it. Each entry ends with the question an examiner is most likely to ask about that piece of code, and the answer.

Part Two is the backend push plan: which member commits which server file, so the repository history matches the contribution table.

Who owns what

Member	Sprint lead	Features owned
Mohammed Adnan Chowdhury	Sprint 1	Bookmark and access Safety Resources and tips (FR-21, FR-22) Modify or monitor reports (Administrator) (FR-25 (part)) Submit Feedback and Report Issues (FR-23) View Incident Analytics (Administrator) (FR-24) Handle and manage Users (Administrator) (FR-25)
Adnan Foysal Shibly	Sprint 2	Manage User Profile (FR-1) Add and Remove emergency Contacts (FR-5) Share Live Location During SOS (FR-2, FR-3) Send Mail SOS Alerts (FR-4) Safety Check-In (FR-26)
Promit Debnath	Sprint 3	React to Community Reports (FR-11) Report Inappropriate Content (FR-12) Save Safe Places (FR-19) Moderate Community Content (FR-13) Receive Safe Place Notifications (FR-20)
Ahmad Said	Sprint 4	Create, Invite, and Leave Safety Groups (FR-14) Group Messaging (FR-15) Share Location with Group Members (FR-16) Receive Group Emergency Alerts (FR-17) Find Nearby Police Stations, Hospitals, and Pharmacies (FR-18)

Member	Sprint lead	Features owned
Sami Bin Hedayet	Sprint 5	Report Safety Incidents and Upload Incident Evidence (FR-6) Store and Categorize Incident Database (FR-7) View Community Safety Map (FR-8) Search Incident Reports, View Details and Comment (FR-9) Maintain SOS History (FR-10)

Part One — The code behind each feature

Twenty-five feature assignments from the contribution tracker, grouped by member. Server files are shown in bold monospace, client files in light monospace.

1 Bookmark and access Safety Resources and tips

FR-21, FR-22

What it does. A small library of safety guides, legal rights and helpline numbers that a user can read and save for later.

How it works. Resources are seeded into the database and served publicly, so they can be read without signing in. A bookmark is polymorphic - the same table stores a saved resource and a saved incident - which is why the unique index is on (user, targetType, targetId) rather than on a resource id.

File	What it contributes
<code>server/src/models/Resource.js</code>	Article schema: title, slug, category, body, pinned flag.
<code>server/src/models/Bookmark.js</code>	Polymorphic save. Unique per (user, targetType, targetId).
<code>server/src/controllers/resourceController.js</code>	listResources, getResource (by id or slug), createResource / updateResource / deleteResource (admin only), addBookmark, listBookmarks, removeBookmark.
<code>server/src/routes/resourceRoutes.js</code>	Public reads; writes gated behind restrictTo('admin').
<code>client/src/pages/resources/Resources.jsx</code>	Category filter and keyword search over the library.
<code>client/src/pages/resources/ResourceDetail.jsx</code>	One article, with a save button.
<code>client/src/pages/resources/Bookmarks.jsx</code>	Everything the user has saved.

Likely question. Why is Bookmark one collection instead of two? Because a user saves resources and incidents, and one polymorphic table keeps a single unique index doing the duplicate-prevention work for both.

2 Modify or monitor reports (Administrator)

FR-25 (part)

What it does. Lets a moderator or admin verify a community report, or reject one that is wrong.

How it works. Verification is a status change on the incident, not a delete. An incident moves between pending / verified / rejected / removed, and the map shows a tick only on verified ones. The handler lives in moderationController but is mounted on the incident routes, because the thing being changed is an incident.

File	What it contributes
<code>server/src/controllers/moderationController.js</code>	setIncidentStatus - the only function in this file owned here; the rest belongs to FR-13.
<code>server/src/routes/incidentRoutes.js</code>	PATCH /incidents/:id/status, restricted to staff.
<code>server/src/config/constants.js</code>	INCIDENT_STATUS: pending, verified, rejected, removed.
<code>client/src/pages/incidents/IncidentDetail.jsx</code>	The staff-only verify / reject controls.

Likely question. Why not delete a false report? Because the audit trail and the reporter's own view still need it. Status is reversible; a delete is not.

What it does. A direct line from the user to the project team for bugs, suggestions and complaints.

How it works. Submission is open to signed-out visitors too, so someone who cannot log in can still report that fact. The submitter gets an acknowledgement email; admins get a triage list with a status field.

File	What it contributes
<code>server/src/models/Feedback.js</code>	type, subject, message, status, optional user link.
<code>server/src/controllers/feedbackController.js</code>	submitFeedback, listMyFeedback, listAllFeedback (admin), respondToFeedback.
<code>server/src/routes/feedbackRoutes.js</code>	Public POST; admin-only list and triage.
<code>client/src/pages/Feedback.jsx</code>	The form, plus the user's own past submissions.

Likely question. Why allow anonymous submission? The most important bug report is 'I cannot sign in', and requiring a session would make it impossible to send.

4 View Incident Analytics (Administrator)

What it does. Dashboards showing how many incidents were reported, in which categories, where, and how the numbers move over time.

How it works. Four MongoDB aggregation pipelines. The hotspot one rounds coordinates to two decimal places and groups on them, which turns scattered points into repeatedly-reported areas, then reverse-geocodes the centre so the admin sees a place name rather than numbers.

File	What it contributes
<code>server/src/controllers/adminController.js</code>	getDashboard (headline counts and trend), getCategoryBreakdown, getTrends (daily series), getHotspots (grouped by rounded coordinates).
<code>server/src/routes/adminRoutes.js</code>	/dashboard and the three /analytics/* routes.
<code>client/src/pages/admin/AdminDashboard.jsx</code>	Stat tiles, category bars and the trend chart.

Likely question. Why round the coordinates for hotspots? Two reports on the same street are never at identical coordinates, so grouping on the raw values would produce a list of ones.

What it does. Suspend or reinstate an account, change a role, and read the audit trail.

How it works. Suspending increments the user's tokenVersion, which invalidates every JWT already issued to them - so the lockout is immediate rather than waiting for a token to expire. A timed suspension lifts itself on the next request. Every action here is written to the audit log.

File	What it contributes
server/src/controllers/adminController.js	setStatus, setRole, getUserDetail, listAuditLogs, getMailQueueStatus, retryFailedMail.
server/src/models/AuditLog.js	action, actor, target, severity, metadata.
server/src/services/auditService.js	record / recordAsync, used across the whole app.
client/src/pages/admin/AdminUsers.jsx	Search, suspend, reinstate, change role.
client/src/pages/admin/AdminAudit.jsx	Filterable audit trail.

Likely question. How does a suspension take effect immediately when JWTs are stateless? tokenVersion is embedded in the token and compared against the user record on every request.

1 Manage User Profile

FR-1

What it does. Account creation, sign-in, and the profile that emergency responders read - name, phone, blood group, medical notes, address, photo.

How it works. Passwords are bcrypt-hashed by a pre-save hook. The access token is short-lived and kept in memory only; the refresh token is an httpOnly cookie, so a script injected into the page cannot steal a session. What each audience may see is decided by userView, not by each controller remembering to strip fields.

File	What it contributes
server/src/models/User.js	Schema, password hashing, passwordChangedAfter, tokenVersion.
server/src/controllers/authController.js	register, login, refresh, logout, logoutAll, forgotPassword, resetPassword, changePassword.
server/src/controllers/userController.js	getProfile, updateProfile, updateAvatar, updatePreferences, searchUsers, getPublicProfile, deactivateAccount.
server/src/middleware/auth.js	protect, optionalAuth, restrictTo - the four ways a token can be stale.
server/src/utils/tokens.js	Sign and verify, random tokens, hashing, constant-time compare.
server/src/views/userView.js	self / publicProfile / adminRow projections.
server/src/routes/authRoutes.js	Auth endpoints and their rate limits.
server/src/routes/userRoutes.js	Profile endpoints.
client/src/pages/Profile.jsx	Tabbed profile: personal, medical, privacy, security.
client/src/pages/auth/Login.jsx	Sign in by email or username.
client/src/pages/auth/Register.jsx	Account creation with live password strength.
client/src/pages/auth/ForgotPassword.jsx	Requests a reset email.
client/src/pages/auth/ResetPassword.jsx	Completes the reset.
client/src/pages/auth/AuthShell.jsx	Shared layout for the four auth screens.
client/src/context/AuthContext.jsx	Session state and the boot-time refresh.

Likely question. Why is the access token never in localStorage? Anything in localStorage is readable by injected script. In memory it dies with the tab, and the httpOnly refresh cookie restores the session on reload.

What it does. The list of people who get emailed when an SOS is raised.

How it works. A compound unique index on (owner, email) stops the same person being added twice. A contact can be switched off without being deleted, which keeps the alert history readable. The cap of ten is enforced in the controller and stated in the UI.

File	What it contributes
<code>server/src/models/EmergencyContact.js</code>	Schema, unique (owner, email), activeForOwner, notifyCount and lastNotifiedAt.
<code>server/src/controllers/contactController.js</code>	listContacts (ordered by priority), createContact, updateContact, deleteContact.
<code>server/src/routes/contactRoutes.js</code>	CRUD with field-level validation.
<code>client/src/pages/Contacts.jsx</code>	Add, edit, switch off, remove; shows the cap.

Likely question. Why keep a contact you have switched off? So a past alert can still say who was notified at the time.

3 Share Live Location During SOS

FR-2, FR-3

What it does. The one-tap alarm, and the moving map the contacts watch afterwards.

How it works. The SOS document is written and the response returns before any email work starts, so the phone never waits on an SMTP handshake. The browser then pushes a point every ten seconds and each one is appended to a GeoJSON trail. The tracking token is stored as a hash and expires; the plain token exists only in the email.

File	What it contributes
<code>server/src/models/SosEvent.js</code>	Trail of GeoJSON points, notifiedContacts, tracking token hash, appendTrailPoint.
<code>server/src/controllers/sosController.js</code>	activateSos, updateLocation, getTrackingByToken, revokeTracking.
<code>server/src/services/sosService.js</code>	activate and appendLocation - the ordering decisions live here.
<code>server/src/views/sosView.js</code>	publicTracking - what a contact holding the link may see.
<code>server/src/routes/sosRoutes.js</code>	Public /track/:token; everything else behind protect.
<code>client/src/context/SosContext.jsx</code>	Activation, the ten-second ticker, the offline point buffer.
<code>client/src/components/sos/SosButton.jsx</code>	Press-and-hold button with a progress ring.
<code>client/src/pages/sos/ActiveSos.jsx</code>	Live screen: trail, delivery status, 'I am safe'.
<code>client/src/pages/sos/TrackSos.jsx</code>	The public page a contact opens from the email.
<code>client/src/hooks/useGeolocation.js</code>	Wraps the browser API; fast fix first, precise second.

Likely question. Why does the SOS respond before the emails are sent? Because a person in danger must not wait on an SMTP handshake. The mail queue makes the alert durable anyway.

What it does. The email itself - who needs help, where they are, their medical details, and a live tracking link.

How it works. Nothing calls SMTP directly. Callers enqueue a MailJob and a worker drains the queue with exponential backoff, so an alert survives a crash or an SMTP outage. Delivery runs four at a time, because sending one by one made the time to reach everybody the sum of every round trip. Every value interpolated into the HTML is escaped.

File	What it contributes
<code>server/src/services/mailService.js</code>	enqueue, deliver, processQueue with bounded concurrency, mirrorToSosEvent.
<code>server/src/models/MailJob.js</code>	Durable queue: status, attempts, backoff, claimNext, requeueStuck.
<code>server/src/views/emails/templates.js</code>	sosAlert, sosResolved, groupSosAlert, checkInDue and the rest.
<code>server/src/views/emails/layout.js</code>	Shared email shell; escapes every value it renders.
<code>server/src/jobs/index.js</code>	The cron worker that drains the queue and requeues stuck jobs.
<code>server/src/config/clientUrl.js</code>	Resolves the address the emailed links point at.

Likely question. What happens if the server dies between raising the SOS and sending the mail? The MailJob row is already in the database, so the next worker tick sends it.

5 Safety Check-In

What it does. A timer set before travelling. If it runs out and the user does not confirm they are safe, the emergency procedure runs by itself.

How it works. Two timestamps and a scheduler: dueAt is when the app asks, escalateAt is when it stops waiting. The important decision is that escalation calls the same sosService.activate the red button calls, so a missed check-in produces an identical alert - same email, same tracking link, same group fan-out - rather than a weaker one.

File	What it contributes
<code>server/src/models/SafetyCheckIn.js</code>	dueAt, graceMinutes, escalateAt, status, extendBy.
<code>server/src/services/checkInService.js</code>	start, confirmSafe, extend, cancel, promptDue, escalateOverdue, runDueChecks.
<code>server/src/controllers/checkInController.js</code>	The seven HTTP handlers.
<code>server/src/routes/checkInRoutes.js</code>	Routes and validation bounds.
<code>server/src/views/checkInView.js</code>	Adds secondsUntilDue so the client countdown cannot drift.
<code>client/src/pages/CheckIn.jsx</code>	Set the timer, live countdown, I am safe / extend / cancel.
<code>server/tests/checkins.test.js</code>	28 tests covering the prompt and the escalation.

Likely question. Why reuse sosService.activate instead of writing a new alert? The requirement says 'the predefined emergency procedure', and there is exactly one of those.

1 React to Community Reports

FR-11

What it does. Helpful / important / support reactions, so useful reports rise to the top.

How it works. Reactions live in an array on the incident, one entry per user, so changing your mind replaces rather than adds. Denormalised counts are recomputed on write so the feed never has to unwind the array to sort.

File	What it contributes
<code>server/src/models/Incident.js</code>	reactions array, reactionCounts, recalculateReactionCounts.
<code>server/src/controllers/incidentController.js</code>	reactToIncident - one reaction per user per report.
<code>server/src/routes/incidentRoutes.js</code>	POST /incidents/:id/react.
<code>client/src/pages/incidents/IncidentDetail.jsx</code>	The three reaction buttons and their counts.

Likely question. Why store both the array and the counts? The array is the truth (who reacted); the counts exist so the feed can sort without unwinding it.

2 Report Inappropriate Content

FR-12

What it does. Lets any member flag a fake, offensive or abusive post or comment.

How it works. A unique index on (reporter, targetType, targetId) means one person can only flag one item once - re-flagging updates the existing row instead of creating a second. Reporting your own content is refused with a message telling you to delete it instead.

File	What it contributes
<code>server/src/models/ContentReport.js</code>	reason, target, status, reviewer, unique per reporter.
<code>server/src/controllers/moderationController.js</code>	reportContent - the filing half of moderation.
<code>server/src/routes/adminRoutes.js</code>	POST /admin/reports, open to any signed-in user.
<code>client/src/pages/incidents/IncidentDetail.jsx</code>	The report dialog on posts and comments.

Likely question. Why is POST /admin/reports open to everybody when the rest of /admin is not? Filing a flag is a user action; reading the queue is a moderator one.

What it does. Named circles the user trusts - home, university, a friend's flat.

How it works. Each place is a centre plus a radius, indexed with 2dsphere. The radius floor is 50 m because consumer GPS is rarely better than that and a tighter circle would flap between inside and outside. A unique index on (owner, label) stops two 'Home's.

File	What it contributes
<code>server/src/models/SafePlace.js</code>	Point, radiusMeters with bounds, notify flags, isInside.
<code>server/src/controllers/placeController.js</code>	listSafePlaces, createSafePlace, updateSafePlace, deleteSafePlace.
<code>server/src/routes/placeRoutes.js</code>	The /safe-places routes.
<code>client/src/pages/SafePlaces.jsx</code>	Map picker, radius slider, notification toggles.

Likely question. Why a minimum radius of 50 m? Below that, normal GPS noise alone would trigger arrive/leave alerts while the user sat still.

What it does. The moderator queue: review a flag, remove or restore content, warn or suspend the author.

How it works. Resolving one flag resolves every other open flag on the same item, so a moderator handles a post once rather than five times. Restore is the one action allowed on an already-reviewed report, because undoing a removal is a decision that can only be taken after the fact.

File	What it contributes
<code>server/src/controllers/moderationController.js</code>	listReports, getReportDetail, resolveReport (dismiss / remove-content / restore-content / warn-user / suspend-user).
<code>server/src/routes/adminRoutes.js</code>	The moderator-only queue routes.
<code>client/src/pages/admin/AdminModeration.jsx</code>	Queue, review dialog, undo-removal path.

Likely question. Why can restore-content run on a closed report when nothing else can? Removing content closes the report, so requiring an open one would make a mistaken removal permanent.

What it does. Tells the user - and optionally their family - when they arrive at or leave a saved place.

How it works. The client posts its position every couple of minutes and the server compares it against every saved fence. A transition is only emitted when the remembered inside/outside state actually flips, which is what stops a stream of duplicate alerts while someone sits at home.

File	What it contributes
<code>server/src/models/SafePlaceEvent.js</code>	enter / leave, with the distance at the time.
<code>server/src/controllers/placeController.js</code>	checkLocation (the geofence evaluator), listSafePlaceEvents.
<code>server/src/services/notificationService.js</code>	notify / notifyMany, shared with SOS and groups.
<code>server/src/models/Notification.js</code>	The in-app feed.
<code>client/src/context/SosContext.jsx</code>	The two-minute geofence ticker (paused during an SOS).

Likely question. Why does the geofence tick pause during an SOS? The SOS ticker already reports position far more often; running both just drains the battery.

1 Create, Invite, and Leave Safety Groups

FR-14

What it does. Private circles - family, flatmates, classmates - that are alerted together.

How it works. Members are a subdocument array with roles, so membership is one read. An invitation is a code stored on the group with an expiry, and the accept link goes out by email and as an in-app notification. Leaving as the owner transfers ownership rather than orphaning the group. A stranger asking for a group gets 404, not 403 - a 403 would confirm it exists.

File	What it contributes
<code>server/src/models/SafetyGroup.js</code>	members with roles, invites, createInvite, isMember.
<code>server/src/controllers/groupController.js</code>	createGroup, listGroups, getGroup, updateGroup, inviteMember, previewInvite, respondToInvite, revokeInvite, leaveGroup, removeMember, setMemberRole, deleteGroup.
<code>server/src/routes/groupRoutes.js</code>	All group routes.
<code>server/src/views/groupView.js</code>	summary / detail / invitation projections.
<code>client/src/pages/groups/Groups.jsx</code>	List and create.
<code>client/src/pages/groups/GroupDetail.jsx</code>	Chat, live map, members, settings tabs.
<code>client/src/pages/groups/GroupInvite.jsx</code>	Preview and accept or decline.

Likely question. Why 404 rather than 403 for a group you are not in? A 403 tells a stranger the group exists, which is itself a leak.

2 Group Messaging

FR-15

What it does. A chat shared by everyone in a safety group.

How it works. Messages are stored and also pushed over Socket.IO, which is authenticated with the same access token as the REST API and puts each socket in a room per group. The message types include system and sos, so an alert is a permanent entry in the conversation rather than a toast that disappears.

File	What it contributes
<code>server/src/models/GroupMessage.js</code>	type (text / location / system / sos), body, sender.
<code>server/src/controllers/groupController.js</code>	listMessages, sendMessage, muteGroup.
<code>server/src/sockets/index.js</code>	JWT handshake, room join/leave, typing indicators.
<code>server/src/sockets/emitter.js</code>	emitToUser / emitToGroup / emitToSos, no-ops without a server.
<code>client/src/context/SocketContext.jsx</code>	Socket with a polling fallback behind one subscribe() API.
<code>client/src/pages/groups/GroupDetail.jsx</code>	The chat tab.

Likely question. Why is an SOS also a chat message? So somebody who opens the app an hour later still sees that it happened.

What it does. Lets a member show the group where they are, for as long as they choose.

How it works. Sharing is opt-in and per group, held on the member subdocument with a timestamp. Stopping removes the position immediately rather than letting a stale point linger, which is the privacy half of the feature.

File	What it contributes
<code>server/src/controllers/groupController.js</code>	<code>shareLocation</code> , <code>stopSharingLocation</code> , <code>getGroupLocations</code> .
<code>server/src/models/SafetyGroup.js</code>	<code>sharedLocation</code> on each member entry.
<code>client/src/pages/groups/GroupDetail.jsx</code>	The live map tab and the share toggle.
<code>client/src/components/map/MapView.jsx</code>	Shared Leaflet wrapper used by every map screen.

Likely question. What happens to a shared position when you stop sharing? It is cleared, not just hidden - the endpoint stops returning it at all.

4 Receive Group Emergency Alerts

What it does. When one member raises an SOS, every other member of every group they belong to is told.

How it works. Fan-out reaches each member three ways: an in-app notification (instant, cannot bounce), a permanent sos message in the group chat, and an email - unless that member has switched group emails off. Members can opt out without leaving the group.

File	What it contributes
<code>server/src/services/sosService.js</code>	The FR-17 block inside <code>fanOut</code> , and the resolve fan-out.
<code>server/src/services/notificationService.js</code>	<code>notifyMany</code> , used for the group broadcast.
<code>server/src/views/emails/templates.js</code>	<code>groupSosAlert</code> .
<code>server/src/models/SafetyGroup.js</code>	<code>alertMembersOnSos</code> , and per-member notification prefs.

Likely question. Why three channels rather than just email? An in-app notification is instant and cannot bounce; email reaches somebody who does not have the app open.

What it does. The closest emergency services to wherever the user is, with distances and directions.

How it works. Data comes from OpenStreetMap's Overpass API, which allows only two concurrent queries per client - so all three categories are fetched in one query and split by tag afterwards, rather than three in parallel where one would always be dropped. Results are cached on a rounded coordinate key so a whole neighbourhood shares one upstream call, and several mirrors are tried before giving up.

File	What it contributes
<code>server/src/services/geoService.js</code>	<code>findNearbyMany</code> (one query, many categories), mirror fallback, Nominatim throttle.
<code>server/src/services/cache.js</code>	TTL cache used for POIs and geocoding.
<code>server/src/controllers/placeController.js</code>	<code>findNearby</code> , <code>findAllNearby</code> , <code>searchPlaces</code> , <code>reverseGeocode</code> .
<code>server/src/utils/geo.js</code>	Haversine distance, bounding boxes, distance formatting.
<code>client/src/pages/Nearby.jsx</code>	Category tabs, radius, map, and the area-search fallback.

Likely question. Why one Overpass query instead of three? Overpass allows two concurrent slots, so three parallel requests reliably lost one - a category would come back empty for no visible reason.

1 Report Safety Incidents and Upload Incident Evidence

FR-6

What it does. Lets a member report harassment, theft or an unsafe area, with photo, video or audio evidence.

How it works. Uploads are checked against their contents, not the type the client claims: the leading bytes are matched against known signatures, so an executable renamed to .png is refused and deleted. If validation fails after the files are written, the controller cleans them up rather than leaving orphans on disk.

File	What it contributes
<code>server/src/models/Incident.js</code>	The report schema, media subdocuments, 2dsphere index.
<code>server/src/controllers/incidentController.js</code>	<code>createIncident</code> , <code>updateIncident</code> , <code>deleteIncident</code> .
<code>server/src/middleware/upload.js</code>	Multer storage, magic-byte signature check, <code>persistUpload</code> , cleanup on failure.
<code>server/src/routes/incidentRoutes.js</code>	POST <code>/incidents</code> with the upload middleware.
<code>client/src/pages/incidents/ReportIncident.jsx</code>	The form, map pin and file previews.

Likely question. Why check the file's bytes when it already declares a MIME type? Because the client writes that type - it is a claim, not a fact.

2 Store and Categorize Incident Database

FR-7

What it does. The nine categories every report is filed under, and the indexes that make them searchable.

How it works. Categories live in one constants file so the model, the validators, the seed data and the client's dropdown can never drift apart - the client fetches them from `/api/meta` rather than keeping its own copy. The collection carries a 2dsphere index for the map and a weighted text index for search.

File	What it contributes
<code>server/src/config/constants.js</code>	INCIDENT_CATEGORIES, labels, severities, statuses - the single source of truth.
<code>server/src/models/Incident.js</code>	Category enum, 2dsphere index, weighted text index.
<code>server/src/routes/index.js</code>	GET <code>/api/meta</code> , which hands every enum to the client.

Likely question. Why does the client fetch the category list instead of hard-coding it? So adding a category is a one-file change rather than two that can disagree.

What it does. An interactive map of everything reported nearby, so a user can see which areas to avoid.

How it works. Pins are fetched for a centre and radius using the 2dsphere index, capped, and returned through a slim mapPin projection rather than the full report - a map of five hundred incidents should not ship five hundred descriptions. Leaflet renders OpenStreetMap tiles, so there is no API key.

File	What it contributes
<code>server/src/controllers/incidentController.js</code>	getMapPins, plus the shared geo filter.
<code>server/src/views/incidentView.js</code>	mapPin - the deliberately minimal projection.
<code>client/src/pages/SafetyMap.jsx</code>	The map screen: category legend, time range, area search.
<code>client/src/components/map/MapView.jsx</code>	Leaflet wrapper, markers, recentring without remounting.

Likely question. Why a separate mapPin projection? The map needs a position, a category and a severity; sending the full description for every pin would be wasteful.

What it does. Finding reports by keyword, area or distance, reading one in full, and discussing it.

How it works. Search uses the weighted text index, so a match in the title outranks one buried in the description. Regex metacharacters in the search term are escaped, which stops `.*` turning into a full collection scan. Removed and rejected reports disappear for everyone except staff and the author.

File	What it contributes
<code>server/src/controllers/incidentController.js</code>	listIncidents (keyword / category / radius), getIncident, listComments, addComment, deleteComment.
<code>server/src/models/Comment.js</code>	Comment schema with soft removal.
<code>server/src/utils/query.js</code>	Pagination clamps, keywordFilter with regex escaping, sort whitelist.
<code>server/src/views/incidentView.js</code>	summary, detail and comment projections.
<code>client/src/pages/incidents/IncidentFeed.jsx</code>	Search, filters and pagination.
<code>client/src/pages/incidents/IncidentDetail.jsx</code>	Full report, evidence, comment thread.

Likely question. What stops a user searching `.*` and scanning the whole collection? keywordFilter escapes regex metacharacters before building the query.

What it does. A record of every past alert - when, how long, where, and who was reached.

How it works. Resolving an alert records the duration and revokes the tracking link in the same step, so a closed alert cannot still be followed. Per-recipient delivery status is mirrored onto the event as the mail queue drains, which is what lets the history say '2 of 2 reached' rather than guessing. Alerts nobody closes are expired by an hourly job so the banner does not follow the user forever.

File	What it contributes
<code>server/src/controllers/sosController.js</code>	<code>listSosHistory</code> , <code>getSosDetail</code> , <code>getAlertStatus</code> , <code>resolveSos</code> , <code>resendAlerts</code> .
<code>server/src/services/sosService.js</code>	<code>resolve</code> (duration, revoke, all-clear) and <code>expireStale</code> .
<code>server/src/views/sosView.js</code>	summary and detail, including the delivery tallies.
<code>client/src/pages/sos/SosHistory.jsx</code>	Filterable list with duration and delivery counts.

Likely question. Why does resolving revoke the tracking token? Otherwise a link forwarded from an old email would keep showing a live position after the emergency ended.

Part Two — GitHub push plan (backend)

Server-side files only. Push in sprint order so the history reads as the project was built: scaffold first, then each member's feature work, then the tests.

Step 1 — Shared backend files

These belong to no single feature. They are assigned by closest affinity so that every member has scaffolding in their history, and the reason is given in case it is asked.

Mohammed Adnan Chowdhury

Sprint 1 leader - opens the repository with the project scaffold

- `package.json`
- `package-lock.json`
- `.gitignore`
- `README.md`
- `PLAN.md`
- `server/package.json`
- `server/nodemon.json`
- `server/.env.example`

Adnan Foysal Shibly

Owns authentication, so the request pipeline and error contract sit with him

- `server/src/server.js`
- `server/src/app.js`
- `server/src/config/env.js`
- `server/src/config/database.js`
- `server/src/config/logger.js`
- `server/src/config/tunnelFile.js`
- `server/src/middleware/errorHandler.js`
- `server/src/middleware/rateLimiter.js`
- `server/src/utils/AppError.js`
- `server/src/utils/apiResponse.js`
- `server/src/utils/asyncHandler.js`
- `server/src/utils/background.js`
- `server/src/utils/sanitize.js`

Promit Debnath

Owns two moderation features, which lean hardest on validation and presenters

- `server/src/validators/validate.js`
- `server/src/views/present.js`
- `server/src/views/commonView.js`
- `server/src/routes/index.js`
- `server/src/routes/notificationRoutes.js`

Ahmad Said

Owns the realtime surface, so the socket gateway and its emitter sit with him

- `server/src/sockets/index.js`
- `server/src/sockets/emitter.js`
- `server/src/services/cache.js`

Sami Bin Hedayet

Owns the incident surface, which the seed data and operational scripts serve

server/src/scripts/seed.js

server/src/scripts/backup.js

server/src/scripts/localdb.js

server/src/scripts/mailcheck.js

server/src/scripts/tunnel.js

Step 2 — Feature files, by member

One commit per feature keeps the history readable and makes each member's contribution obvious at a glance. A file listed under two members is shared: whoever pushes first creates it, the second amends it, and the commit message should say which functions were added.

Mohammed Adnan
Chowdhury

Commit	Backend files to push
Sprint 1 Bookmark and access Safety Resources and tips	<code>server/src/models/Resource.js</code> <code>server/src/models/Bookmark.js</code> <code>server/src/controllers/resourceController.js</code> <code>server/src/routes/resourceRoutes.js</code>
Sprint 2 Modify or monitor reports (Administrator)	<code>server/src/controllers/moderationController.js</code> <code>server/src/routes/incidentRoutes.js</code> <code>server/src/config/constants.js</code>
Sprint 3 Submit Feedback and Report Issues	<code>server/src/models/Feedback.js</code> <code>server/src/controllers/feedbackController.js</code> <code>server/src/routes/feedbackRoutes.js</code>
Sprint 4 View Incident Analytics (Administrator)	<code>server/src/controllers/adminController.js</code> <code>server/src/routes/adminRoutes.js</code>
Sprint 5 Handle and manage Users (Administrator)	<code>server/src/controllers/adminController.js</code> <code>server/src/models/AuditLog.js</code> <code>server/src/services/auditService.js</code>
Tests push last, once the features are in	<code>server/tests/admin.test.js</code>

Commit	Backend files to push
Sprint 1 Manage User Profile	server/src/models/User.js server/src/controllers/authController.js server/src/controllers/userController.js server/src/middleware/auth.js server/src/utils/tokens.js server/src/views/userView.js server/src/routes/authRoutes.js server/src/routes/userRoutes.js
Sprint 2 Add and Remove emergency Contacts	server/src/models/EmergencyContact.js server/src/controllers/contactController.js server/src/routes/contactRoutes.js
Sprint 3 Share Live Location During SOS	server/src/models/SosEvent.js server/src/controllers/sosController.js server/src/services/sosService.js server/src/views/sosView.js server/src/routes/sosRoutes.js
Sprint 4 Send Mail SOS Alerts	server/src/services/mailService.js server/src/models/MailJob.js server/src/views/emails/templates.js server/src/views/emails/layout.js server/src/jobs/index.js server/src/config/clientUrl.js
Sprint 5 Safety Check-In	server/src/models/SafetyCheckIn.js server/src/services/checkInService.js server/src/controllers/checkInController.js server/src/routes/checkInRoutes.js server/src/views/checkInView.js server/tests/checkins.test.js
Tests push last, once the features are in	server/tests/auth.test.js server/tests/contacts.test.js server/tests/sos.test.js server/tests/checkins.test.js

Commit	Backend files to push
Sprint 1 React to Community Reports	server/src/models/Incident.js server/src/controllers/incidentController.js server/src/routes/incidentRoutes.js
Sprint 2 Report Inappropriate Content	server/src/models/ContentReport.js server/src/controllers/moderationController.js server/src/routes/adminRoutes.js
Sprint 3 Save Safe Places	server/src/models/SafePlace.js server/src/controllers/placeController.js server/src/routes/placeRoutes.js
Sprint 4 Moderate Community Content	server/src/controllers/moderationController.js server/src/routes/adminRoutes.js
Sprint 5 Receive Safe Place Notifications	server/src/models/SafePlaceEvent.js server/src/controllers/placeController.js server/src/services/notificationService.js server/src/models/Notification.js
Tests push last, once the features are in	server/tests/moderation.test.js server/tests/places.test.js

Commit	Backend files to push
Sprint 1 Create, Invite, and Leave Safety Groups	server/src/models/SafetyGroup.js server/src/controllers/groupController.js server/src/routes/groupRoutes.js server/src/views/groupView.js
Sprint 2 Group Messaging	server/src/models/GroupMessage.js server/src/controllers/groupController.js server/src/sockets/index.js server/src/sockets/emitter.js
Sprint 3 Share Location with Group Members	server/src/controllers/groupController.js server/src/models/SafetyGroup.js
Sprint 4 Receive Group Emergency Alerts	server/src/services/sosService.js server/src/services/notificationService.js server/src/views/emails/templates.js server/src/models/SafetyGroup.js
Sprint 5 Find Nearby Police Stations, Hospitals, and Pharmacies	server/src/services/geoService.js server/src/services/cache.js server/src/controllers/placeController.js server/src/utils/geo.js
Tests push last, once the features are in	server/tests/groups.test.js server/tests/security.test.js

Commit	Backend files to push
Sprint 1 Report Safety Incidents and Upload Incident Evidence	server/src/models/Incident.js server/src/controllers/incidentController.js server/src/middleware/upload.js server/src/routes/incidentRoutes.js
Sprint 2 Store and Categorize Incident Database	server/src/config/constants.js server/src/models/Incident.js server/src/routes/index.js
Sprint 3 View Community Safety Map	server/src/controllers/incidentController.js server/src/views/incidentView.js
Sprint 4 Search Incident Reports, View Details and Comment	server/src/controllers/incidentController.js server/src/models/Comment.js server/src/utils/query.js server/src/views/incidentView.js
Sprint 5 Maintain SOS History	server/src/controllers/sosController.js server/src/services/sosService.js server/src/views/sosView.js
Tests push last, once the features are in	server/tests/incidents.test.js server/tests/units.test.js server/tests/helpers.js server/tests/setup.js

Step 3 — Commands

Each member works on their own branch and opens a pull request, so the history shows who wrote what and nobody overwrites anybody.

Once, by the Sprint 1 leader

```
git init
git branch -M main
git remote add origin https://github.com/<org>/nirapod.git
git add package.json .gitignore README.md PLAN.md server/package.json
git commit -m "chore: project scaffold"
git push -u origin main
```

Then, by every member in turn

```
git checkout main && git pull
git checkout -b feature/<your-feature>
git add <the files listed for you above>
git commit -m "feat(FR-n): <feature name>"
git push -u origin feature/<your-feature>
```

Before pushing, always

```
npm test # 295 tests must pass
npm run build # the client must still build
```